

SISTEM PROGRAMLAMA DERSİ LAB5 RAPORU

Öğrenci Adı: Bartu Şanlı

Öğrenci Numarası: 23220030065

Açıklayıcı Video Linki:

https://drive.google.com/file/d/11b_3tq5zfKv0bESyd24Jz9O2E35fYNSM/view?usp=sharing

Konu: UNIX/Linux Sistemlerde Süreçler Arası İletişim (IPC) Mekanizmaları: Pipe ve FIFO Uygulamaları

GİRİŞ

Bu laboratuvar çalışması, UNIX/Linux ortamında süreçler arası iletişim (IPC) sağlamak için kullanılan iki temel mekanizma olan **Adsız Pipe (Unnamed Pipe)** ve **Adlandırılmış Pipe (Named Pipe / FIFO)** yapılarının C programlama dili kullanılarak incelenmesi ve uygulanmasını kapsamaktadır.

Çalışmanın amacı, pipe(), fork(), read(), write(), dup2(), execlp() ve mkfifo() gibi sistem çağrılarını kullanarak:

1. Akriba süreçler arasında tek ve çift yönlü iletişimi (Adsız Pipe),
2. İlişkisiz süreçler arasında İstemci/Sunucu modelini (Adlandırılmış Pipe / FIFO),

gerçekleştirmek ve gözlemlemektir.

BÖLÜM 1: ADSIZ PIPE (UNNAMED PIPE) UYGULAMALARI

Adsız Pipe'lar, yalnızca birbirleriyle akraba (genellikle Parent-Child) olan süreçler arasında **tek yönlü** veri akışı sağlayan bir IPC yöntemidir. İletişim için kullanılan dosya tanımlayıcıları (fd[0] ve fd[1]), pipe() sistem çağrısı ile oluşturulur.

1.1. Adım 1 & 2: Shell Pipe Simülasyonu (getent | grep)

Bu adımda, kabukta kullanılan boru (|) operatörü işlevi, dup2() sistemi kullanılarak simüle edilmiştir. getent komutunun standart çıktısı, pipe'ın yazma ucuna yönlendirilmiş, grep komutunun standart girdisi ise pipe'ın okuma ucuna yönlendirilmiştir.

Kullanılan Kod (pipe_sim.c):

```
#include <stdio.h>
#include <stdlib.h> // exit() için [cite: 19]
#include <unistd.h> // fork(), pipe(), dup2(), execlp() için [cite: 19]
#include <sys/wait.h> // wait() için

// Parent-Child arası IPC (getent group | grep kullanıcı_adi)
int main(void) {
    int fd[2]; /* Adsız pipe için iki tanımlayıcı dizisi: fd[0] okuma, fd[1] yazma */
    int pid1, pid2; /* Süreç tanımlayıcıları [cite: 20] */
    const char *kullanıcı_adi = "bartu_sanli"; // <<<<< BURAYI DEĞİŞTİRİN

    /* Pipe, fork()'tan ÖNCE oluşturulmalıdır ki, child süreçler kopyasına >
    if (pipe(fd) < 0) {
        perror("PIPE OLUŞTURMA HATASI");
        exit(1);
    }

    // --- BİRİNCİ ÇOCUK SÜREÇ (GREP) OLUŞTURULUYOR (Pipe'tan okur) ---
    pid1 = fork();
    if (pid1 < 0) {
        perror("ILK FORK HATASI");
        exit(1);
    }

    if (pid1 == 0) {
        // 1. Child: grep

        // Pipe'ın okuma ucunu (fd[0]) standart girişe (0) yönlendir.
        // Böylece grep komutu pipe'tan okuma yapacaktır. [cite: 23]
        dup2(fd[0], 0);

        // Bu süreç pipe'ın yazma ucunu kullanmayacak, bu yüzden kapatılır. >
        close(fd[1]);

        // Bu süreç pipe'ın okuma ucunu da kullanmayacak (çünkü dup2 ile 0' >
        close(fd[0]);
```

```

GNU nano 7.2                                pipe_sim.c
// "grep kullanıcı_adi" komutu çalıştırılır. [cite: 25]
// execlp, başarılı olursa bu süreci grep ile değiştirir.
execlp("grep", "grep", kullanıcı_adi, (char *)0);

// execlp başarısız olursa buradan devam eder
perror("GREP HATA");
exit(1);
}

// --- İKİNCİ ÇOCUK SÜREÇ (GETENT) OLUŞTURULUYOR (Pipe'a yazar) ---
// Parent süreç devam ediyor ve ikinci çocuğu oluşturuyor [cite: 26]
pid2 = fork();
if (pid2 < 0) {
    perror("İKİNCİ FORK HATASI");
    exit(1);
}

if (pid2 == 0) {
    // 2. Child: getent
    // Pipe'ın yazma ucunu (fd[1]) standart çıkışa (1) yönlendir.
    // Böylece getent komutunun çıktısı pipe'a yazılacaktır. [cite: 27]
    dup2(fd[1], 1);

    // Bu süreç pipe'ın okuma ucunu kullanmayacak, bu yüzden kapatılır.
    close(fd[0]);

    // Bu süreç pipe'ın yazma ucunu da kullanmayacak (çünkü dup2 ile 1'
    close(fd[1]);

    // "getent group" komutu çalıştırılır. [cite: 29]
    execlp("getent", "getent", "group", (char *)0);

    // execlp başarısız olursa buradan devam eder
    perror("GETENT HATA");
    exit(1);
}

// --- ANA SÜREÇ (PARENT) DEVAM EDİYOR ---
// Parent, pipe'ın kendi uçlarını kapatır [cite: 30]
close(fd[0]);
close(fd[1]);

// Her iki çocuğun da sonlanmasını bekler [cite: 30]
waitpid(pid1, NULL, 0); // grep'in bitmesini bekle
waitpid(pid2, NULL, 0); // getent'in bitmesini bekle

printf("Ana Süreç Tamamlandı.\n");
return 0;
}

```

Komutlar ve Çıktı:

Komut	Amaç	Çıktı (Örnek)
gcc pipe_sim.c -o pipe_sim	Derleme	

Komut	Amaç	Çıktı (Örnek)
./pipe_sim	Program Çıktısı	Kendi kullanıcı adınızın olduğu grup satırları
`getent group`	grep [kullanıcı_adınız]`	Shell Komutu

```
bartu_sanli@bartu-computer:~/lab5a$ ./pipe_sim
adm:x:4:syslog,bartu_sanli
cdrom:x:24:bartu_sanli
sudo:x:27:bartu_sanli
dip:x:30:bartu_sanli
plugdev:x:46:bartu_sanli
users:x:100:bartu_sanli
bartu_sanli:x:1000:
Ana Süreç Tamamlandı.
```

```
bartu_sanli@bartu-computer:~/lab5a$ getent group
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:syslog,bartu_sanli
tty:x:5:
disk:x:6:
lp:x:7:
mail:x:8:
news:x:9:
uucp:x:10:
man:x:12:
proxy:x:13:
kmem:x:15:
dialout:x:20:
fax:x:21:
voice:x:22:
cdrom:x:24:bartu_sanli
floppy:x:25:
tape:x:26:
sudo:x:27:bartu_sanli
audio:x:29:
dip:x:30:bartu_sanli
www-data:x:33:
backup:x:34:
operator:x:37:
list:x:38:
irc:x:39:
src:x:40:
shadow:x:42:
utmp:x:43:
video:x:44:
sasl:x:45:
plugdev:x:46:bartu_sanli
staff:x:50:
games:x:60:
users:x:100:bartu_sanli
nogroup:x:65534:
systemd-journal:x:999:
systemd-network:x:998:
```

```
crontab:x:997:
systemd-timesync:x:996:
input:x:995:
sgx:x:994:
kvm:x:993:
render:x:992:
messagebus:x:101:
syslog:x:102:
systemd-resolve:x:991:
uuidd:x:103:
_ssh:x:104:
landscape:x:105:
polkitd:x:990:
admin:x:106:
netdev:x:107:
bartu_sanli:x:1000:
```

```
bartu_sanli@bartu-computer:~/lab5a$ getent group | grep bartu_sanli
adm:x:4:syslog,bartu_sanli
cdrom:x:24:bartu_sanli
sudo:x:27:bartu_sanli
dip:x:30:bartu_sanli
plugdev:x:46:bartu_sanli
users:x:100:bartu_sanli
bartu_sanli:x:1000:
```

Yorum:

Program çıktısı, kabuk komutunun çıktısı ile birebir aynıdır. **dup2()** çağırısı, dosya tanımlayıcılarını başarılı bir şekilde yeniden atamıştır: getent sürecinde standart çıktı (1) pipe'ın yazma ucuna atanmış, grep sürecinde standart girdi (0) pipe'ın okuma ucuna atanmıştır. Böylece, iki farklı sürecin çıktı ve girdileri arasında tek yönlü iletişim sağlanmıştır.

1.2. Adım 3 & 4: Manuel write() ve read() Kullanımı

Bu adımda harici komutlar yerine, mesajlar doğrudan pipe'a **write()** ile yazılmış ve **read()** ile okunmuştur.

Kullanılan Kod (pipe_rw.c):


```
#include <stdio.h>
#include <stdlib.h> // exit() için
#include <unistd.h> // fork(), pipe(), close(), read(), write() için
#include <sys/wait.h> // wait() için

// Parent-Child arası IPC (Manual write/read)
int main(void) {
    int fd[2]; /* Adsız pipe için tanımlayıcı dizisi: fd[0] okuma, fd[1] y>
    int pid1, pid2; /* Süreç tanımlayıcıları */

    /* Pipe, fork()'tan ÖNCE oluşturulmalıdır. */
    if (pipe(fd) < 0) {
        perror("PIPE OLUŞTURMA HATASI");
        exit(1);
    }

    // --- BİRİNCİ ÇOCUK SÜREÇ (OKUYUCU) OLUŞTURULUYOR ---
    pid1 = fork();
    if (pid1 < 0) {
        perror("ILK FORK HATASI");
        exit(1);
    }

    if (pid1 == 0) {
        // 1. Child: Reader (Okuyucu)
        char buf[100];
        ssize_t nbytes; // read() fonksiyonu ssize_t tipinde değer döndürür

        // Okuyucu, pipe'ın yazma ucunu KULLANMAYACAKTIR.
        close(fd[1]);

        printf("Okuyucu (%d): Pipe'tan okumaya basliyorum...\n", getpid());

        // Pipe'tan okuma işlemi. read(), okunan bayt sayısını döndürür. 0 >
        while ((nbytes = read(fd[0], buf, sizeof(buf) - 1)) > 0) {
            // Okunan veriyi ekrana (Standart Çıkış = 1) yazdır.
            write(1, buf, nbytes);
        }
    }
}
```

```

        // Okuma işi bitti, pipe'ın okuma ucunu kapat.
        close(fd[0]);

        printf("Okuyucu (%d): Okuma işlemi tamamlandı, cikiyorum.\n", getpid());
        exit(0);
    }

    // --- İKİNCİ ÇOCUK SÜREÇ (YAZICI) OLUŞTURULUYOR ---
    // Parent süreç devam ediyor ve ikinci çocuğu oluşturuyor
    pid2 = fork();
    if (pid2 < 0) {
        perror("İKİNCİ FORK HATASI");
        exit(1);
    }

    if (pid2 == 0) {
        // 2. Child: Writer (Yazıcı)

        // Yazıcı, pipe'ın okuma ucunu KULLANMAYACAKTIR.
        close(fd[0]);

        printf("Yazici (%d): Mesajlari pipe'a yaziyorum...\n", getpid());
        // Yazma işlemleri: write(dosya_tanimlayicisi, buffer, boyut)

        // Verilen kelimeleri pipe'ın yazma ucuna yazma:
        write(fd[1], "this\n", 5);
        write(fd[1], "is\n", 3);
        write(fd[1], "a\n", 2);
        write(fd[1], "message\n", 8);
        write(fd[1], "from\n", 5);
        write(fd[1], "sending\n", 8);
        write(fd[1], "process\n", 8);

        // Yazma işi bitti, pipe'ın yazma ucunu kapat.
        // Bu, Okuyucu sürece verinin bittiğini (EOF) bildirir.
        close(fd[1]);
    }

```

```

        printf("Yazici (%d): Yazma işlemi tamamlandı, cikiyorum.\n", getpid());
        exit(0);
    }

    // --- ANA SÜREÇ (PARENT) DEVAM EDİYOR ---
    // Parent, pipe'ın kendi uçlarını kapatır
    close(fd[0]);
    close(fd[1]);

    // Her iki çocuğun da sonlanmasını bekler
    waitpid(pid1, NULL, 0); // Okuyucunun bitmesini bekle
    waitpid(pid2, NULL, 0); // Yazıcının bitmesini bekle

    printf("Ana Süreç Tamamlandı.\n");
    return 0;
}

```


Çıktı:

```
bartu_sanli@bartu-computer:~/lab5a$ ./pipe_rw
Okuyucu (721): Pipe'tan okumaya basliyorum...
Yazici (722): Mesajlari pipe'a yaziyorum...
this
is
a
message
from
sending
Yazici (722): Yazma islemi tamamlandi, cikiyorum.
process
Okuyucu (721): Okuma islemi tamamlandi, cikiyorum.
Ana Süreç Tamamlandı.
```

Yorum:

Yazıcı süreç, `write(fd[1], ...)` kullanarak veriyi pipe'a göndermiş, Okuyucu süreç ise bir döngü içinde `read(fd[0], ...)` ile veriyi almıştır. Kritik nokta, Yazıcı sürecin işi bittikten sonra **`close(fd[1])`** çağrısı yapmasıdır. Bu çağrı, Okuyucu sürece **EOF (End of File)** sinyali göndermiş ve Okuyucunun `read()` döngüsünden düzgünce çıkmasını sağlamıştır. Bu kapatma işlemi olmasaydı, Okuyucu süreç veri beklerken **bloke** kalacaktı.

1.3. Adım 5: Çoklu Yazma ve Atomiklik Testi

Bu adımda, iki ayrı yazıcı süreç (Yazıcı 1 ve Yazıcı 2) aynı anda aynı pipe'a veri yazmış, üçüncü bir süreç (Okuyucu) ise bu verileri okumuştur.

Kullanılan Kod (pipe_atomicity.c):

```

GNU nano 7.2                                pipe_atomicity.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MESSAGE_SIZE 50

// Bir çocuk süreç için mesaj yazma fonksiyonu
void writer_process(int write_fd, int id, const char *message) {
    char full_message[MESSAGE_SIZE];

    // Mesajı hazırlama
    snprintf(full_message, MESSAGE_SIZE, "Mesaj %d: %s (PID: %d)\n", id, me>
    size_t len = strlen(full_message);

    printf("Yazici %d basladi, mesaji gonderiyor... (Uzunluk: %zu)\n", id, >

    // Mesajı pipe'a yazma
    ssize_t written_bytes = write(write_fd, full_message, len);

    if (written_bytes != (ssize_t)len) {
        perror("WRITE HATASI");
    } else {
        printf("Yazici %d: %zd byte basariyla yazildi.\n", id, written_byte>
    }

    exit(0);
}

// Okuyucu süreç
void reader_process(int read_fd) {
    char buf[200];
    ssize_t nbytes;

    printf("Okuyucu basladi, pipe'tan okumayi bekliyor... (PID: %d)\n", get>

```

```
// Pipe'tan veri okuma döngüsü
// read(), okuyacak veri kalmadığında ve tüm yazma uçları kapandığında >
while ((nbytes = read(read_fd, buf, sizeof(buf) - 1)) > 0) {
    // Okunan veriyi null sonlandır ve ekrana yazdır.
    buf[nbytes] = '\0';
    printf("Okuyucu: %zd byte alındı. Mesaj:\n%s", nbytes, buf);
}

if (nbytes == 0) {
    printf("Okuyucu: Tüm yazma uçları kapandı, okuma bitti.\n");
} else {
    perror("READ HATASI");
}

exit(0);
}

int main(void) {
    int fd[2]; // fd[0] okuma, fd[1] yazma
    pid_t pid_w1, pid_w2, pid_r;

    if (pipe(fd) < 0) {
        perror("PIPE OLUSTURMA HATASI");
        exit(1);
    }

    // --- 1. ÇOCUK SÜREÇ (YAZICI 1) ---
    pid_w1 = fork();
    if (pid_w1 == 0) {
        close(fd[0]); // Yazıcı okuma ucunu kapatır
        writer_process(fd[1], 1, "İlk mesajım.");
    } else if (pid_w1 < 0) {
        perror("Yazıcı 1 Fork Hatası");
        exit(1);
    }
}
```

```

// --- 2. ÇOCUK SÜREÇ (YAZICI 2) ---
pid_w2 = fork();
if (pid_w2 == 0) {
    close(fd[0]); // Yazıcı okuma ucunu kapatır
    writer_process(fd[1], 2, "ikinci mesajım, daha uzun.");
} else if (pid_w2 < 0) {
    perror("Yazıcı 2 Fork Hatası");
    exit(1);
}

// --- 3. ÇOCUK SÜREÇ (OKUYUCU) ---
pid_r = fork();
if (pid_r == 0) {
    close(fd[1]); // Okuyucu yazma ucunu kapatır
    reader_process(fd[0]);
} else if (pid_r < 0) {
    perror("Okuyucu Fork Hatası");
    exit(1);
}

// --- ANA SÜREÇ (PARENT) ---
// Parent, kendi pipe uçlarını kapatır (çünkü iletişime karışmayacak)
close(fd[0]);
close(fd[1]);

// Tüm çocukların bitmesini bekle
waitpid(pid_w1, NULL, 0);
waitpid(pid_w2, NULL, 0);
waitpid(pid_r, NULL, 0);

printf("Ana Süreç: Tüm çocuklar sonlandı. Program sonu.\n");
return 0;
}

```

Çıktı :

```

bartu_sanli@bartu-computer:~/lab5a$ ./pipe_atomicity
Yazıcı 1 başladı, mesajı gönderiyor... (Uzunluk: 33)
Yazıcı 1: 33 byte başarıyla yazıldı.
Yazıcı 2 başladı, mesajı gönderiyor... (Uzunluk: 47)
Yazıcı 2: 47 byte başarıyla yazıldı.
Okuyucu başladı, pipe'tan okumayı bekliyor... (PID: 841)
Okuyucu: 80 byte alındı. Mesaj:
Mesaj 1: İlk mesajım. (PID: 839)
Mesaj 2: İkinci mesajım, daha uzun. (PID: 840)
Okuyucu: Tüm yazma uçları kapandı, okuma bitti.
Ana Süreç: Tüm çocuklar sonlandı. Program sonu.

```


Yorum:

İki yazıcı sürecin mesajları, okuyucu süreç tarafından **tam ve bütün** olarak alınmıştır (Mesaj 1 ve Mesaj 2 karışmamıştır). Bu durum, UNIX/Linux sistemlerde pipe'lara yazma işlemlerinin (belirli bir boyuta kadar) **atomik** olduğunu doğrular. Yani, bir süreç pipe'a yazmaya başladığında, bu işlem tamamlanana kadar diğer süreçler bloke edilir ve veri bütünlüğü sağlanır. Mesajların ekrana basılma sırası rastgele olabilir (işletim sisteminin planlayıcısına bağlıdır), ancak mesajların kendisi bölünmez.

1.4. Adım 6: Çift Yönlü İletişim (Echo Programı)

Adsız Pipe'ların tek yönlü kısıtlamasını aşmak için iki ayrı pipe kullanılarak Parent ve Child arasında çift yönlü iletişim (Echo) kurulmuştur.

Kullanılan Kod (pipe_echo.c):

```
GNU nano 7.2 pipe_echo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_MSG_SIZE 100
#define TEST_MESSAGE "Merhaba, bu bir yankılama (echo) mesajıdır."

// Pipe Tanımlayıcıları:
// fd_pc: Parent'tan Child'a (pc) iletişim için pipe
// fd_cp: Child'dan Parent'a (cp) iletişim için pipe
int fd_pc[2]; // fd_pc[0]: Okuma ucu (Child kullanır), fd_pc[1]: Yazma ucu >
int fd_cp[2]; // fd_cp[0]: Okuma ucu (Parent kullanır), fd_cp[1]: Yazma ucu >

void parent_process(void) {
    char read_buf[MAX_MSG_SIZE];
    ssize_t nbytes;

    printf("PARENT: Başlıyor (PID: %d)\n", getpid());

    // Parent, Child'dan Okuma (fd_cp[0]) ve Child'a Yazma (fd_pc[1]) uçları >
    // Kullanılmayan uçları kapat.
    close(fd_pc[0]); // Parent, Child'dan okumayı kullanmaz
    close(fd_cp[1]); // Parent, Child'a yazmayı kullanmaz

    printf("PARENT: Mesajı Child'a gönderiyor: \"%s\"\n", TEST_MESSAGE);

    // Parent, Pipe 1'e yazar
    write(fd_pc[1], TEST_MESSAGE, strlen(TEST_MESSAGE) + 1); // +1 null kar >

    // Parent, Pipe 1'in yazma ucunu kapatır (Child'a mesajın bittiğini bil >
    close(fd_pc[1]);

    // Parent, Pipe 2'den Child'ın yolladığı yankı mesajını okur
    printf("PARENT: Child'dan yankılanan mesajı bekliyor...\n");
    nbytes = read(fd_cp[0], read_buf, MAX_MSG_SIZE);
}
```



```
if (nbytes > 0) {
    // Okunan veriyi ekrana yazdır
    printf("PARENT: %zd byte alındı. YANKI MESAJI: \"%s\\n\", nbytes, r>
} else if (nbytes == 0) {
    printf("PARENT: Child pipe'i kapattı, veri gelmedi.\\n");
} else {
    perror("PARENT READ HATASI");
}

// Parent, Pipe 2'nin okuma ucunu kapatır
close(fd_cp[0]);

// Child'ın bitmesini bekle
wait(NULL);
printf("PARENT: Tamamlandı.\\n");
}

void child_process(void) {
    char read_buf[MAX_MSG_SIZE];
    ssize_t nbytes;

    printf("CHILD: Başlıyor (PID: %d)\\n", getpid());

    // Child, Parent'tan Okuma (fd_pc[0]) ve Parent'a Yazma (fd_cp[1]) uçla>
    // Kullanılmayan uçları kapat.
    close(fd_pc[1]); // Child, Parent'a yazmayı kullanmaz
    close(fd_cp[0]); // Child, Parent'tan okumayı kullanmaz

    // Child, Pipe 1'den Parent'ın mesajını okur
    printf("CHILD: Parent'tan mesajı bekliyor...\\n");
    nbytes = read(fd_pc[0], read_buf, MAX_MSG_SIZE);

    if (nbytes > 0) {
        printf("CHILD: %zd byte alındı. Mesaj: \"%s\\n\", nbytes, read_buf);

        // Child, mesajı Parent'a geri yankılar (Pipe 2'ye yazar)
        printf("CHILD: Mesajı Parent'a geri yankılıyor.\\n");
```

```
        write(fd_cp[1], read_buf, nbytes);
    } else if (nbytes < 0) {
        perror("CHILD READ HATASI");
    }

    // Child, Pipe 1'in okuma ucunu ve Pipe 2'nin yazma ucunu kapatır
    close(fd_pc[0]);
    close(fd_cp[1]);

    printf("CHILD: Tamamlandi.\n");
    exit(0);
}

int main(void) {

    // 1. Parent -> Child (P->C) pipe'ını oluştur
    if (pipe(fd_pc) < 0) {
        perror("P->C PIPE HATASI");
        return 1;
    }

    // 2. Child -> Parent (C->P) pipe'ını oluştur
    if (pipe(fd_cp) < 0) {
        perror("C->P PIPE HATASI");
        return 1;
    }

    // 3. Child sürecini oluştur
    pid_t pid = fork();

    if (pid < 0) {
        perror("FORK HATASI");
        return 1;
    }

    if (pid == 0) {
```

```
        if (pid == 0) {
            // CHILD Süreci
            child_process();
        } else {
            // PARENT Süreci
            parent_process();
        }

        return 0;
    }
}
```

Çıktı:

```
bartu_sanli@bartu-computer:~/lab5a$ ./pipe_echo
PARENT: Basliyor (PID: 850)
PARENT: Mesaji Child'a gonderiyor: "Merhaba, bu bir yankilama (echo) mesajid
ir."
PARENT: Child'dan yankilanan mesaji bekliyor...
CHILD: Basliyor (PID: 851)
CHILD: Parent'tan mesaji bekliyor...
CHILD: 44 byte alindi. Mesaj: "Merhaba, bu bir yankilama (echo) mesajidir."
CHILD: Mesaji Parent'a geri yankiliyor.
CHILD: Tamamlandi.
PARENT: 44 byte alindi. YANKI MESAJI: "Merhaba, bu bir yankilama (echo) mesa
jidir."
PARENT: Tamamlandi.
```

Yorum:

Her akış yönü için (Parent'tan Child'a ve Child'dan Parent'a) ayrı bir pipe kullanılarak iki yönlü iletişim sağlanmıştır. Pipe'ların kullanılmayan uçları kapatılarak akış güvenliği sağlanmıştır. Bu kurulum, akraba süreçler arasında tam **dubleks** veri akışının nasıl sağlanacağını göstermektedir.

BÖLÜM 2: ADLANDIRILMIŞ PIPE (FIFO) UYGULAMALARI

Adlandırılmış Pipe (FIFO), dosya sisteminde bir isimle var olan ve **ilişkisiz süreçler** arasında tek yönlü iletişim sağlayan bir IPC mekanizmasıdır.

2.1. Adım 1-5: Basit İstemci/Sunucu ve Girdi Yönlendirme

Bu adımlarda myfifo adlı tek bir FIFO kullanılarak basit İstemci/Sunucu iletişimi gözlemlenmiştir.

```
bartu_sanli@bartu-computer:~/lab5b$ ./server &
[1] 899
```

2.2. Adım 7: İki Yönlü İstemci/Sunucu Uygulaması

Bu adımda, tam dubleks iletişim için iki ayrı FIFO (req_pipe ve res_pipe) kullanılarak bir istek/cevap/kapanma akışı gerçekleştirilmiştir.

Kullanılan Kodlar:

```
/* A server using FIFO pipes for two-way communication */

#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h> // exit() için
#include <unistd.h> // unlink(), read(), write(), close() için
#include <string.h> // strcmp() için

#define MAX_BUFFER 1024
#define REPLY_MESSAGE "THIS IS A REPLY FROM SERVER"
#define BYE_MESSAGE "bye"

// Kullanım: ./server pipe_req pipe_res &
int main(int argc, char *argv[]) {
    int fd_req, fd_res;
    ssize_t n_read;
    char buf[MAX_BUFFER];

    if (argc < 3) {
        printf("Kullanım: %s <request_pipe_name> <response_pipe_name> &\n",
            argv[0]);
        exit(1);
    }

    printf("SERVER BASLIYOR... (PID: %d)\n", getpid());

    // 1. Önceki pipe'ları sil (Gerekliyse) [cite: 75]
    unlink(argv[1]);
    unlink(argv[2]);

    // 2. İki pipe'ı oluştur: (argv[1] -> İstemciden İsteği Okuma, argv[2] -> İstemciye Yanıt Yazma)
    // Client Request Pipe (Sunucu okuma, İstemci yazma)
    if (mkfifo(argv[1], 0666) < 0) {
        perror("İSTEK PIPE (REQ) OLUSTURMA HATASI");
        exit(1);
    }
}
```



```
printf("ISTEK PIPE (%s) OLUSTURULDU.\n", argv[1]);  
  
// Server Response Pipe (Sunucu yazma, İstemci okuma)  
if (mkfifo(argv[2], 0666) < 0) {  
    perror("CEVAP PIPE (RES) OLUSTURMA HATASI");  
    // Eğer ikincisi başarısız olursa, ilkinin silmek iyi bir uygulamadır  
    unlink(argv[1]);  
    exit(1);  
}  
printf("CEVAP PIPE (%s) OLUSTURULDU.\n", argv[2]);  
  
// Orijinal kodda izin ayarı vardı, bu adımı isteğe bağlı bırakıyorum:  
// chmod(argv[1], 0666);  
// chmod(argv[2], 0666);  
  
// --- BÖLÜM A: İSTEK ALMA VE CEVAP GÖNDERME ---  
  
// 3. İstemcinin istek göndermesi için İSTEK PIPE'ını (argv[1]) okuma m>  
// Bu open() çağrısı, bir İstemci yazma modunda açana kadar BLOKE olur.  
printf("\nISTEK PIPE'I OKUMA ICIN ACILIYOR (%s) ve Istemci bekleniyor..>  
if ((fd_req = open(argv[1], O_RDONLY)) < 0) {  
    perror("ISTEK PIPE ACMA HATASI (O_RDONLY)");  
    exit(1);  
}  
printf("ISTEK PIPE'I BASARIYLA ACILDI. Istemci baglandi.\n");  
  
// 4. İstemciden gelen ilk mesajı (istek) oku  
printf("ISTEMCI ISTEĞİ OKUNUYOR...\n");  
n_read = read(fd_req, buf, sizeof(buf) - 1);  
  
if (n_read > 0) {  
    buf[n_read] = '\0';  
    printf("ALINAN İSTEK MESAJI: %s\n", buf);  
  
    // 5. Cevap göndermek için CEVAP PIPE'ını (argv[2]) yazma modunda aç  
    // Client'ın bu pipe'ı okuma modunda açması BLOKAJI çözer.  
    if ((fd_res = open(argv[2], O_WRONLY)) < 0) {
```


GNU nano 7.2

server.c *

```
if ((fd_res = open(argv[2], O_WRONLY)) < 0) {
    perror("CEVAP PIPE ACMA HATASI (O_WRONLY)");
    close(fd_req);
    exit(1);
}
printf("CEVAP PIPE'I BASARIYLA ACILDI. Cevap gönderiliyor...\n");

// 6. Cevabı İstemciye yaz
write(fd_res, REPLY_MESSAGE, strlen(REPLY_MESSAGE) + 1);
printf("CEVAP MESAJI (%s) İstemciye gönderildi.\n", REPLY_MESSAGE);

// 7. Cevap pipe'ını kapat
close(fd_res);
}

// 8. İstemcinin "bye" mesajını bekleme adımı
// İlk okuma işlemi tamamlandı, pipe'ın okuma ucunu kapatıp tekrar açma>
// Ancak FIFO'larda tek bir okuma süreci tarafından kapatılan uç, EOF s>
// Bu akış için, ilk mesajı aldıktan sonra İstemci tarafından pipe'ın k>
// tekrar açılmasını bekleyen bir model uygulayacağız.

// İlk istek pipe'ının okuma ucunu kapat (EOF sinyalini aldıktan sonra)
close(fd_req);

// Client'ın "bye" mesajını göndermesi için İSTEK PIPE'ını (argv[1]) te>
// Bu open() çağrısı, client'ın 'bye' mesajını yollamak için bağlanması>
printf("\nCLIENT'IN SONLANMA İSTEGI ('bye') BEKLENİYOR...\n");
if ((fd_req = open(argv[1], O_RDONLY)) < 0) {
    perror("İSTEK PIPE ACMA HATASI (BYE BEKLERKEN)");
    exit(1);
}

// "bye" mesajını oku
n_read = read(fd_req, buf, sizeof(buf) - 1);
if (n_read > 0) {
    buf[n_read] = '\0';
    if (strcmp(buf, BYE_MESSAGE) == 0) {
```

```
// "bye" mesajını oku
n_read = read(fd_req, buf, sizeof(buf) - 1);
if (n_read > 0) {
    buf[n_read] = '\0';
    if (strcmp(buf, BYE_MESSAGE) == 0) {
        printf("CLIENT'TAN 'bye' MESAJI ALINDI. Pipelar siliniyor.\n");
    } else {
        printf("CLIENT'TAN BEKLENMEYEN MESAJ ALINDI: %s\n", buf);
    }
}

// 9. Pipe'ları sil [cite: 74]
unlink(argv[1]);
unlink(argv[2]);

printf("SERVER SONLANDIRILDI.\n");
exit(0);
}
```

```
/* A client using the FIFO pipes for two-way communication */

#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h> // exit() için
#include <unistd.h> // read(), write(), close() için
#include <string.h> // strlen() için

#define MAX_BUFFER 1024
#define REQUEST_MESSAGE "Hi Server, may I get a reply?"
#define BYE_MESSAGE "bye"

// Kullanım: ./client pipe_req pipe_res
int main(int argc, char *argv[]) {
    int fd_req, fd_res;
    ssize_t n_read;
    char buf[MAX_BUFFER];

    if (argc < 3) {
        printf("Kullanım: %s <request_pipe_name> <response_pipe_name>\n", a
        exit(1);
    }

    // argv[1]: İstek Pipe Adı (Sunucu Okur, İstemci Yazar)
    // argv[2]: Cevap Pipe Adı (İstemci Okur, Sunucu Yazar)

    printf("CLIENT BASLADI... (PID: %d)\n", getpid());

    // --- BÖLÜM A: İSTEK GÖNDERME ---

    // 1. İstek göndermek için İSTEK PIPE'ını (argv[1]) yazma modunda aç
    // Bu open() çağrısı, Sunucu open() çağrısının BLOKAJINI çözer.
    printf("ISTEK PIPE'I YAZMA ICIN ACILIYOR (%s)...\n", argv[1]);
    if ((fd_req = open(argv[1], O_WRONLY)) < 0) {
        perror("ISTEK PIPE ACMA HATASI (O_WRONLY)");
    }
}
```

```

GNU nano 7.2                                client.c *
printf("ISTEK PIPE'I ACILDI. Istek gönderiliyor.\n");

// 2. İsteği Sunucuya yaz
write(fd_req, REQUEST_MESSAGE, strlen(REQUEST_MESSAGE) + 1);
printf("ISTEK MESAJI ('%s') Sunucuya gönderildi.\n", REQUEST_MESSAGE);

// 3. İstek pipe'ını kapat (İstek gönderimi tamamlandı)
close(fd_req);

// --- BÖLÜM B: CEVAP ALMA ---

// 4. Cevabı almak için CEVAP PIPE'ını (argv[2]) okuma modunda aç
// Sunucu'nun yazma ucu açıldıktan sonra bu open() çağrısı BLOKE olmaz.
printf("\nCEVAP PIPE'I OKUMA ICIN ACILIYOR (%s) ve Cevap bekleniyor...\n");
if ((fd_res = open(argv[2], O_RDONLY)) < 0) {
    perror("CEVAP PIPE ACMA HATASI (O_RDONLY)");
    exit(1);
}
printf("CEVAP PIPE'I ACILDI. Cevap okunuyor.\n");

// 5. Cevabı oku ve ekrana yazdır
while ((n_read = read(fd_res, buf, sizeof(buf) - 1)) > 0) {
    buf[n_read] = '\0';
    printf("SUNUCUDAN ALINAN CEVAP: %s\n", buf);
}

// 6. Cevap pipe'ını kapat
close(fd_res);

// --- BÖLÜM C: SONLANMA İSTEĞİ (BYE) GÖNDERME ---

// 7. Pipe'ları silmesi için Sunucuya 'bye' mesajını tekrar İSTEK PIPE'I
// Pipe'ı tekrar yazma modunda aç.
printf("\nSUNUCUYA KAPANMA İSTEĞİ ('%s') GONDERILİYOR...\n", BYE_MESSAGE);
if ((fd_req = open(argv[1], O_WRONLY)) < 0) {
    perror("ISTEK PIPE ACMA HATASI (BYE ICIN)");
    exit(1);
}

// 8. 'bye' mesajını yaz ve pipe'ı kapat
write(fd_req, BYE_MESSAGE, strlen(BYE_MESSAGE) + 1);
close(fd_req);

printf("CLIENT SONLANDIRILDI.\n");
return 0;
}

```

Çalıştırma Komutları ve Akış Analizi:

1. Terminal 1 (Sunucu):


```

bartu_sanli@bartu-computer:~/lab5b$ ./server req_pipe res_pipe
SERVER BASLIYOR... (PID: 927)
ISTEK PIPE (req_pipe) OLUSTURULDU.
CEVAP PIPE (res_pipe) OLUSTURULDU.

ISTEK PIPE'I OKUMA ICIN ACILIYOR (req_pipe) ve Istemci bekleniyor...
./client req_pipe res_pipe
Bu bir denemedir
ISTEK PIPE'I BASARIYLA ACILDI. Istemci baglandi.
ISTEMCI ISTEĞİ OKUNUYOR...
ALINAN ISTEK MESAJI: Hi Server, may I get a reply?
CEVAP PIPE'I BASARIYLA ACILDI. Cevap gönderiliyor...
CEVAP MESAJI (THIS IS A REPLY FROM SERVER) Istemciye gönderildi.

CLIENT'IN SONLANMA ISTEĞİ ('bye') BEKLENİYOR...
CLIENT'TAN 'bye' MESAJI ALINDI. Pipelar siliniyor.
SERVER SONLANDIRILDI.
bartu_sanli@bartu-computer:~/lab5b$ ./client req_pipe res_pipe
CLIENT BASLADI... (PID: 961)
ISTEK PIPE'I YAZMA ICIN ACILIYOR (req_pipe)...
ISTEK PIPE ACMA HATASI (O_WRONLY): No such file or directory
bartu_sanli@bartu-computer:~/lab5b$ Bu bir denemedir

```

2. Terminal 2 (İstemci):

```

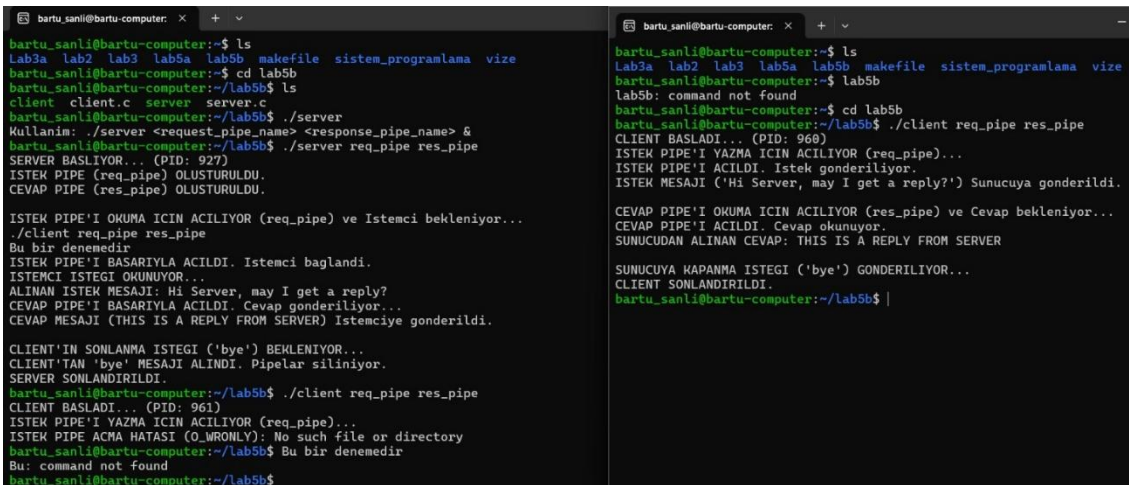
bartu_sanli@bartu-computer:~/lab5b$ ./client req_pipe res_pipe
CLIENT BASLADI... (PID: 960)
ISTEK PIPE'I YAZMA ICIN ACILIYOR (req_pipe)...
ISTEK PIPE'I ACILDI. Istek gönderiliyor.
ISTEK MESAJI ('Hi Server, may I get a reply?') Sunucuya gönderildi.

CEVAP PIPE'I OKUMA ICIN ACILIYOR (res_pipe) ve Cevap bekleniyor...
CEVAP PIPE'I ACILDI. Cevap okunuyor.
SUNUCUDAN ALINAN CEVAP: THIS IS A REPLY FROM SERVER

SUNUCUYA KAPANMA ISTEĞİ ('bye') GONDERİLİYOR...
CLIENT SONLANDIRILDI.

```

3. Genel Bakış:



```

bartu_sanli@bartu-computer: ~$ ls
Lab3a  lab2  lab3  lab5a  lab5b  makefile  sistem_programlama  vize
bartu_sanli@bartu-computer: ~$ cd lab5b
bartu_sanli@bartu-computer: ~/lab5b$ ls
client  client.c  server  server.c
bartu_sanli@bartu-computer: ~/lab5b$ ./server
Kullanım: ./server <request_pipe_name> <response_pipe_name> &
bartu_sanli@bartu-computer: ~/lab5b$ ./server req_pipe res_pipe
SERVER BASLIYOR... (PID: 927)
ISTEK PIPE (req_pipe) OLUSTURULDU.
CEVAP PIPE (res_pipe) OLUSTURULDU.

ISTEK PIPE'I OKUMA ICIN ACILIYOR (req_pipe) ve Istemci bekleniyor...
./client req_pipe res_pipe
Bu bir denemedir
ISTEK PIPE'I BASARIYLA ACILDI. Istemci baglandi.
ISTEMCI ISTEĞİ OKUNUYOR...
ALINAN ISTEK MESAJI: Hi Server, may I get a reply?
CEVAP PIPE'I BASARIYLA ACILDI. Cevap gönderiliyor...
CEVAP MESAJI (THIS IS A REPLY FROM SERVER) Istemciye gönderildi.

CLIENT'IN SONLANMA ISTEĞİ ('bye') BEKLENİYOR...
CLIENT'TAN 'bye' MESAJI ALINDI. Pipelar siliniyor.
SERVER SONLANDIRILDI.
bartu_sanli@bartu-computer: ~/lab5b$ ./client req_pipe res_pipe
CLIENT BASLADI... (PID: 961)
ISTEK PIPE'I YAZMA ICIN ACILIYOR (req_pipe)...
ISTEK PIPE ACMA HATASI (O_WRONLY): No such file or directory
bartu_sanli@bartu-computer: ~/lab5b$ Bu bir denemedir
Bu: command not found
bartu_sanli@bartu-computer: ~/lab5b$

```

```

bartu_sanli@bartu-computer: ~$ ls
Lab3a  lab2  lab3  lab5a  lab5b  makefile  sistem_programlama  vize
bartu_sanli@bartu-computer: ~$ cd lab5b
bartu_sanli@bartu-computer: ~/lab5b$ ./client req_pipe res_pipe
CLIENT BASLADI... (PID: 960)
ISTEK PIPE'I YAZMA ICIN ACILIYOR (req_pipe)...
ISTEK PIPE'I ACILDI. Istek gönderiliyor.
ISTEK MESAJI ('Hi Server, may I get a reply?') Sunucuya gönderildi.

CEVAP PIPE'I OKUMA ICIN ACILIYOR (res_pipe) ve Cevap bekleniyor...
CEVAP PIPE'I ACILDI. Cevap okunuyor.
SUNUCUDAN ALINAN CEVAP: THIS IS A REPLY FROM SERVER

SUNUCUYA KAPANMA ISTEĞİ ('bye') GONDERİLİYOR...
CLIENT SONLANDIRILDI.
bartu_sanli@bartu-computer: ~/lab5b$

```

Yorum:

İki yönlü iletişim, her akış yönü için ayrı bir FIFO (req_pipe ve res_pipe) kullanılarak sağlanmıştır. FIFO'ların **blokaj** özelliği, Sunucunun İstemciyi beklemesini ve Client-Server senkronizasyonunu mümkün kılmıştır. Sunucunun son adımda unlink() kullanarak FIFO'ları silmesi, dosya sisteminde kalıntı bırakılmasını engellemiştir.

SONUÇ

Bu laboratuvar çalışması, Adsız Pipe ve Adlandırılmış Pipe (FIFO) IPC mekanizmalarının temel işleyişini, uygulama farklılıklarını ve kullanım amaçlarını başarıyla göstermiştir. Bu uygulamalar, modern işletim sistemlerinde çoklu görevli programların geliştirilmesi ve birbirleriyle senkronize çalışması için kritik öneme sahiptir.